

Analizador Léxico (Lexer)

Visão Geral

O analisador léxico é composto por três módulos que trabalham em sequência:



A separação entre pré-processador e lexer foi uma decisão deliberada para respeitar a estrutura peculiar do formato fixo do Fortran 77, onde as regras de colunas e continuação de linha são responsabilidade do pré-processador e não do reconhecimento de tokens.

Módulo `tokens.py`

Define os contratos partilhados entre o pré-processador, o lexer e o parser: a tupla `TOKENS`, o conjunto `KEYWORDS` e a string `LITERALS`.

Categorias de tokens

Categoria	Tokens
Estrutura do programa	<code>PROGRAM</code> , <code>END</code> , <code>STOP</code> , <code>RETURN</code>
Subprogramas	<code>FUNCTION</code> , <code>SUBROUTINE</code> , <code>CALL</code>
Tipos de dados	<code>INTEGER</code> , <code>REAL</code> , <code>LOGICAL</code> , <code>CHARACTER</code>
Controlo de fluxo	<code>IF</code> , <code>THEN</code> , <code>ELSE</code> , <code>ENDIF</code> , <code>DO</code> , <code>CONTINUE</code> , <code>GOTO</code>
I/O	<code>READ</code> , <code>PRINT</code>
Funções intrínsecas	<code>MOD</code> , <code>SQRT</code> , <code>MAX</code> , <code>MIN</code>
Identificadores	<code>IDEN</code>
Labels	<code>LABEL</code>
Literais	<code>INT_LIT</code> , <code>REAL_LIT</code> , <code>STRING_LIT</code>
Operadores relacionais	<code>OP_EQ</code> , <code>OP_NE</code> , <code>OP_LT</code> , <code>OP_LE</code> , <code>OP_GT</code> , <code>OP_GE</code>
Operadores lógicos	<code>OP_AND</code> , <code>OP_OR</code> , <code>OP_NOT</code>
Booleanos	<code>TRUE</code> , <code>FALSE</code>

Categoria	Tokens
Potência	POWER (* *)
Marcador de fim de linha	NEWLINE

Os **LITERALS** (+- */=(,),) são caracteres individuais tratados diretamente pelo PLY sem necessidade de regras explícitas.

Decisão de design: **KEYWORDS** como **frozenset**

As keywords são guardadas num **frozenset** para garantir $O(1)$ nas verificações de pertença. O lexer identifica primeiro qualquer sequência **[A-Z][A-Z0-9_]*** como **IDEN** e depois verifica se o valor pertence a **KEYWORDS**, reclassificando o token se necessário — técnica comum em lexers PLY para evitar conflitos com regras de identificadores.

Módulo **preprocessor.py**

Formato Fixo do Fortran 77

O Fortran 77 usa formato fixo de 80 colunas com semântica posicional:

Colunas	Índices Python	Significado
1	[0]	C ou * → linha de comentário
1–5	[0:5]	Label numérico (opcional)
6	[5]	Carácter de continuação (não-espaco e não-0)
7–72	[6:72]	Código Fortran executável
73–80	[72:]	Ignorado (zona de identificação de cartão)

Esta é uma herança do tempo dos cartões perfurados (IBM 80-column punch cards). O pré-processador abstrai toda esta complexidade antes de o lexer PLY ver o código.

Estrutura **LogicalLine**

```
@dataclass
class LogicalLine:
    label: str | None # label extraído das colunas 1-5
    content: str # código extraído das colunas 7-72
    src_line: int # número da primeira linha física
```

Uma **LogicalLine** pode agregar várias linhas físicas quando existem linhas de continuação. O **src_line** aponta sempre para a primeira linha física do grupo, garantindo que os erros são reportados na linha correta.

Algoritmo de processamento

O método `process()` é uma máquina de estados simples:

1. Ignora linhas vazias e linhas de comentário (coluna 1 = `C` ou `*`).
2. Converte toda a linha para maiúsculas (Fortran 77 é case-insensitive; a conversão antecipada simplifica todas as fases seguintes).
3. Verifica o carácter na coluna 6:
 - Espaço ou `'0'` → nova linha lógica.
 - Outro → linha de continuação: concatena o código ao conteúdo da linha lógica em curso.
4. Ao encontrar uma nova linha lógica, guarda a anterior (se existir) e inicia a nova.
5. No final do ficheiro, guarda a última linha lógica pendente.

Tratamento de erros

- **Label inválido:** um label que não seja puramente numérico (ex: `ABC`) lança `PreprocessorError` imediatamente.
- **Continuação sem antecessor:** linha com carácter de continuação sem linha anterior activa lança `PreprocessorError`.

A conversão para maiúsculas nesta fase foi uma decisão importante: evita que as fases seguintes precisem de lidar com comparações case-insensitive, e é consistente com a norma ANSI X3.9-1978.

Módulo `lexer.py`

Integração com PLY

O `Lexer` usa `ply.lex` como motor de reconhecimento de padrões. A classe `Lexer` define diretamente os métodos `t_*` que o PLY processa por reflexão. O PLY é instanciado uma única vez no `__init__` (`lex.lex(module=self)`), e o mesmo objeto é reutilizado para múltiplos inputs através de `_lexer.input()`.

Pipeline de tokenização

O método `tokenize(src: str)` não invoca o PLY directamente sobre o código-fonte. Em vez disso:

1. Instancia um `Preprocessor` e processa o código → `list[LogicalLine]`.
2. Para cada `LogicalLine`, chama `_tokenize_line()`.
3. `_tokenize_line()` injeta manualmente um token `LABEL` (se existir label) antes de entregar o conteúdo ao PLY, e um token `NEWLINE` sintético no final.
4. Devolve a lista plana de todos os tokens.

Esta abordagem garante que o parser recebe sempre `LABEL` antes do conteúdo da linha e `NEWLINE` no final, tornando a gramática mais simples e regular.

Regras de tokenização

As regras são funções Python com docstring-regex, ordenadas por comprimento decrescente (convenção PLY para resolver ambiguidades):

Padrão	Token	Detalhe
<code>\.EQ\.</code>	<code>OP_EQ</code>	Operadores relacionais estilo Fortran
<code>\.AND\.</code>	<code>OP_AND</code>	Operadores lógicos estilo Fortran
<code>\.TRUE\.</code>	<code>TRUE</code>	Valor convertido para <code>bool</code> Python
<code>**</code>	<code>POWER</code>	Antes de <code>*</code> simples para evitar conflito
<code>\d+\.\d*(?:[ED][-+]?[d+])?</code>	<code>REAL_LIT</code>	Notação científica com <code>E</code> ou <code>D</code>
<code>\d+</code>	<code>INT_LIT</code>	Inteiros; deve vir depois de <code>REAL_LIT</code>
<code>`([^\`])*</code>	<code>"*"</code>	<code>STRING_LIT</code>
<code>[A-Z][A-Z0-9_]*</code>	<code>IDEN</code> / keyword	Reclassificado se pertencer a <code>KEYWORDS</code>

Nota sobre `REAL_LIT`: o padrão suporta notação científica com `D` (precisão dupla, comum em Fortran 77 legado). O valor é convertido para `float` Python substituindo `D` por `e`.

Nota sobre `STRING_LIT`: a regex `'([^\`]|')*'` captura strings com aspas simples escapadas por duplicação (estilo Fortran). O processamento `t.value[1:-1].replace("'", "'')` remove as plicas externas e normaliza as internas.

Tokens sintéticos (LABEL e NEWLINE)

O PLY não tem mecanismo nativo para injetar tokens fora das regras de regex. A solução foi construir `LexToken` manualmente com `_make_token()`:

```
def _make_token(self, *, type_: str, value, lineno: int) -> lex.LexToken:
    tok = lex.LexToken()
    tok.type = type_
    tok.value = value
    tok.lineno = lineno
    tok.lexpos = 0
    return tok
```

O `LABEL` tem como valor o inteiro do label (já convertido), o `NEWLINE` tem o valor `'\n'`. O parser usa o `NEWLINE` como terminador de linha nas produções gramaticais.

Interface com o Parser

O `Lexer` implementa os métodos `input(source)` e `token()` esperados pelo PLY yacc, além de `__iter__` e `__next__` para iteração direta. O parser PLY chama `lexer.input()` e `lexer.token()` internamente.

Tratamento de erros

O método `t_error()` acumula erros em `self._errors` (não lança exceções imediatamente) e avança um carácter com `t.lexer.skip(1)`, tentando continuar a tokenização para reportar o máximo de erros possível numa única passagem. No final, o parser verifica `lexer.errors` e propaga-os.

Considerações sobre o Design Global

- **Separação de responsabilidades:** pré-processador lida com formato físico; lexer lida com tokens. Esta separação facilita a manutenção e o teste isolado.
- **Acumulação de erros:** tanto o **Preprocessor** (por exceção) como o **Lexer** (por lista) reportam erros com número de linha, facilitando o diagnóstico.
- **Conversão antecipada para maiúsculas:** feita no pré-processador, elimina a necessidade de comparações case-insensitive em todas as fases seguintes.
- **Tokens NEWLINE explícitos:** o Fortran 77 é sensível à estrutura de linhas (ao contrário de C, por exemplo). Tornar **NEWLINE** um token de primeira classe simplifica a gramática do parser, que pode usá-lo como terminador em todas as produções de declarações e instruções.